

2.3 Nested Components

Part of Components Fundamentals Module - Duration: 10-12 hours

1. Parent and Child Components

1.1 Component Hierarchy Basics

In Angular, components can contain other components, creating a parent-child relationship. This hierarchical structure is fundamental to building complex applications from smaller, reusable pieces.

Key Concepts:

- **Parent Component:** A component that contains and displays other components
- **Child Component:** A component that is used within another component's template
- **Component Tree:** The hierarchical structure formed by nested components
- **Composition:** Building complex UIs by combining simpler components

```
AppComponent (Parent/Root)
├── HeaderComponent (Child of App, Parent of Logo & Nav)
│   ├── LogoComponent (Child of Header)
│   └── NavigationComponent (Child of Header)
├── ProductListComponent (Child of App, Parent of ProductCard)
│   ├── ProductCardComponent (Child of ProductList)
│   ├── ProductCardComponent (Child of ProductList)
│   └── ProductCardComponent (Child of ProductList)
└── FooterComponent (Child of App)
```

Output:

Relationship:

- AppComponent is the parent of HeaderComponent, ProductListComponent, and FooterComponent
- HeaderComponent is a child of AppComponent but a parent to LogoComponent and NavigationComponent
- ProductCardComponent is a child of ProductListComponent and has no children (leaf component)

1.2 Creating Parent-Child Relationship

1.2.1 Step 1: Create Components

```
// Create parent component
ng g c product-list

// Create child component
ng g c product-card
```

Output:**Generated Structure:**

```
src/app/
├─ product-list/
│   ├─ product-list.component.ts
│   ├─ product-list.component.html
│   └─ product-list.component.css
└─ product-card/
    ├─ product-card.component.ts
    ├─ product-card.component.html
    └─ product-card.component.css
```

1.2.2 Step 2: Define Child Component

```
// product-card.component.ts (Child)
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-product-card',
  templateUrl: './product-card.component.html',
  styleUrls: ['./product-card.component.css']
})
export class ProductCardComponent {
  productName: string = 'Wireless Mouse';
  price: number = 29.99;
  imageUrl: string = 'assets/mouse.jpg';
  inStock: boolean = true;

  onBuyClick(): void {
    console.log(`${this.productName} added to cart`);
  }
}
```

```
<!-- product-card.component.html (Child Template) -->
<div class="product-card">
  <img [src]="imageUrl" [alt]="productName">
  <h3>{{ productName }}</h3>
  <p class="price">${{ price }}</p>
  <p class="stock" [class.in-stock]="inStock" [class.out-stock]="!inStock">
    {{ inStock ? 'In Stock' : 'Out of Stock' }}
  </p>
  <button (click)="onBuyClick()" [disabled]="!inStock">
    Buy Now
  </button>
</div>
```

```
/* product-card.component.css (Child Styles) */
.product-card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 15px;
  text-align: center;
  background-color: white;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  transition: transform 0.2s;
}

.product-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 4px 8px rgba(0,0,0,0.15);
}
```

```
}

.product-card img {
  width: 100%;
  height: 200px;
  object-fit: cover;
  border-radius: 4px;
}

.product-card h3 {
  margin: 10px 0;
  color: #333;
  font-size: 18px;
}

.price {
  font-size: 24px;
  font-weight: bold;
  color: #4CAF50;
  margin: 10px 0;
}

.stock {
  font-size: 14px;
  margin: 5px 0;
}

.in-stock {
  color: #4CAF50;
}

.out-stock {
  color: #f44336;
}

button {
  width: 100%;
  padding: 10px;
  background-color: #2196F3;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 16px;
}

button:hover:not(:disabled) {
```

```
background-color: #0b7dda;
}

button:disabled {
  background-color: #ccc;
  cursor: not-allowed;
}
```

1.2.3 Step 3: Use Child in Parent

```
// product-list.component.ts (Parent)
import { Component } from '@angular/core';

@Component({
  selector: 'app-product-list',
  templateUrl: './product-list.component.html',
  styleUrls: ['./product-list.component.css']
})
export class ProductListComponent {
  pageTitle: string = 'Our Products';
  showProducts: boolean = true;

  toggleProducts(): void {
    this.showProducts = !this.showProducts;
  }
}
```

```
<!-- product-list.component.html (Parent Template) -->
<div class="product-list-container">
  <header>
    <h1>{{ pageTitle }}</h1>
    <button (click)="toggleProducts()">
      {{ showProducts ? 'Hide' : 'Show' }} Products
    </button>
  </header>

  <div class="products-grid" *ngIf="showProducts">
    <!-- Child component used multiple times -->
    <app-product-card></app-product-card>
    <app-product-card></app-product-card>
    <app-product-card></app-product-card>
  </div>
</div>
```

```
<app-product-card></app-product-card>
</div>
</div>
```

```
/* product-list.component.css (Parent Styles) */
.product-list-container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px;
}

header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 30px;
}

header h1 {
  color: #333;
  margin: 0;
}

header button {
  padding: 10px 20px;
  background-color: #4CAF50;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.products-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 20px;
}
```

Output:

Rendered Output:

- Parent component renders with title "Our Products" and toggle button
- Grid layout displays 4 product cards

- Each card is an independent instance of ProductCardComponent
- Each card has its own hover effects, buttons, and functionality
- Clicking toggle button shows/hides all cards simultaneously

1.3 Benefits of Parent-Child Structure

Benefit	Description	Example
Reusability	Child component can be used multiple times	ProductCardComponent used 4 times in grid
Maintainability	Changes to child affect all instances	Update card design once, all cards update
Separation of Concerns	Each component has specific responsibility	Parent handles layout, child handles product display
Testing	Components can be tested independently	Test ProductCard without testing ProductList
Code Organization	Complex UI broken into manageable pieces	E-commerce page split into header, products, footer

2. Rendering Components

2.1 Element Selector (Standard Method)

The most common way to use components is with element selectors. The selector becomes a custom HTML element.

```
// Component definition
@Component({
  selector: 'app-user-card', // Element selector
  template: `
    <div class="user-card">
      <h3>{{ userName }}</h3>
      <p>{{ userEmail }}</p>
    </div>
```

```
})  
export class UserCardComponent {  
  userName: string = 'John Doe';  
  userEmail: string = 'john@example.com';  
}
```

```
<!-- Usage in parent template -->  
<div class="container">  
  <app-user-card></app-user-card>  
  <app-user-card></app-user-card>  
</div>
```

Output:

Browser Output (Simplified DOM):

```
<div class="container">  
  <app-user-card>  
    <div class="user-card">  
      <h3>John Doe</h3>  
      <p>john@example.com</p>  
    </div>  
  </app-user-card>  
  <app-user-card>...</app-user-card>  
</div>
```

⚡ Important:

Best Practice: Element selectors are the recommended and most common way to use components. They are clear, semantic, and easy to understand.

2.2 Attribute Selector

Components can also be used as attributes on existing elements. This is less common but useful in specific scenarios.


```
// Component with attribute selector
@Component({
  selector: '[app-highlight]', // Attribute selector (note the brackets)
  template: `<ng-content></ng-content>`,
  styles: [`
    :host {
      display: block;
      background-color: yellow;
      padding: 10px;
      border: 2px solid orange;
    }
  `]
})
export class HighlightComponent {
  constructor() {
    console.log('Highlight component created');
  }
}
```

```
<!-- Usage as attribute -->
<div app-highlight>
  This content is highlighted by the component
</div>

<p app-highlight>
  This paragraph is also highlighted
</p>
```

Output:

Result: Both the div and paragraph have yellow background and orange border. The component wraps the existing element and applies its styles.

Note:

When to use attribute selectors:

- When you want to enhance existing elements
- For directive-like behavior
- When element selector would create unnecessary nesting
- Less common in typical component development

2.3 Class Selector

Components can be triggered by CSS classes. This is rarely used but available.

```
// Component with class selector
@Component({
  selector: '.app-alert', // Class selector (note the dot)
  template: `
    <div class="alert-box">
      <strong>Alert:</strong>
      <ng-content></ng-content>
    </div>
  `,
  styles: [`
    .alert-box {
      background-color: #ffebee;
      border-left: 4px solid #f44336;
      padding: 15px;
      margin: 10px 0;
    }
  `]
})
export class AlertComponent { }
```

```
<!-- Usage with class -->
<div class="app-alert">
  This is an important message!
</div>
```

Output:

Result: The div is enhanced with the alert styling and structure defined in AlertComponent.

⚡ Important:

Recommendation: Class selectors are discouraged for components. They can be confusing because CSS classes are typically used for styling, not component instantiation. Stick with element selectors unless you have a specific reason to use attributes or classes.

2.4 Selector Comparison

Selector Type	Syntax	Usage	Recommendation
Element	<code>selector: 'app-card'</code>	<code><app-card></app-card></code>	☑ Use for most components
Attribute	<code>selector: '[app-card]'</code>	<code><div app-card></div></code>	⚠ Use for directive-like behavior
Class	<code>selector: '.app-card'</code>	<code><div class="app-card"></div></code>	✗ Avoid, confusing with CSS

3. Component Instances

3.1 Creation and Destruction

Every time you use a component selector in a template, Angular creates a new instance of that component class. Each instance has its own properties, methods, and state.

3.1.1 Component Instance Creation

```
// counter.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
    <div class="counter">
      <h3>Counter: {{ count }}</h3>
      <button (click)="increment()">+1</button>
    </div>
  `
})
```

```

    <button (click)="decrement()">-1</button>
    <button (click)="reset()">Reset</button>
  </div>
`,
  styles: [`
    .counter {
      border: 2px solid #2196F3;
      padding: 15px;
      margin: 10px;
      border-radius: 8px;
      text-align: center;
    }
    button {
      margin: 0 5px;
      padding: 8px 15px;
      cursor: pointer;
    }
  `]
})
export class CounterComponent {
  count: number = 0; // Each instance has its own count

  constructor() {
    console.log('Counter instance created');
  }

  increment(): void {
    this.count++;
    console.log(`Counter value: ${this.count}`);
  }

  decrement(): void {
    this.count--;
    console.log(`Counter value: ${this.count}`);
  }

  reset(): void {
    this.count = 0;
    console.log('Counter reset to 0');
  }

  ngOnDestroy(): void {
    console.log('Counter instance destroyed');
  }
}

```

```
<!-- app.component.html -->
<div class="app-container">
  <h1>Independent Counter Instances</h1>

  <!-- Three separate instances -->
  <app-counter></app-counter>
  <app-counter></app-counter>
  <app-counter></app-counter>
</div>
```

Output:

Console Output When Page Loads:

```
Counter instance created
Counter instance created
Counter instance created
```

Behavior:

- Three completely independent counters are created
- Each has its own `count` property starting at 0
- Clicking +1 on first counter: only first counter's count increases
- Second and third counters remain at 0
- Each counter maintains its own state independently

3.1.2 Instance Independence

```
// Demonstration of instance independence
// User Interaction Example:
```

1. Click +1 on first counter → First shows 1, others show 0
2. Click +1 on first counter 5 times → First shows 5, others show 0
3. Click +1 on second counter 3 times → First shows 5, second shows 3, third shows 0
4. Click Reset on first counter → First shows 0, second shows 3, third shows 0

Output:

Key Concept: Each component instance is completely independent. They don't share properties or state unless explicitly programmed to do so (which we'll cover in component communication).

3.2 Component Lifecycle

Components are created and destroyed based on Angular's rendering logic. Understanding when instances are created and destroyed is crucial.

```
// app.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <div>
      <button (click)="toggleCounter()">
        {{ showCounter ? 'Hide' : 'Show' }} Counter
      </button>

      <!-- Counter created when showCounter is true -->
      <app-counter *ngIf="showCounter"></app-counter>
    </div>
  `
})
export class AppComponent {
  showCounter: boolean = true;

  toggleCounter(): void {
    this.showCounter = !this.showCounter;
  }
}
```

Output:

Lifecycle Flow:

Initial Load:

- `showCounter = true`
- Angular creates CounterComponent instance
- Console: "Counter instance created"

- Counter displays on screen

Click "Hide Counter":

- `showCounter = false`
- Angular destroys CounterComponent instance
- Console: "Counter instance destroyed"
- Counter removed from DOM
- **Important:** Counter's state (count value) is LOST

Click "Show Counter" again:

- `showCounter = true`
- Angular creates NEW CounterComponent instance
- Console: "Counter instance created"
- Counter displays with count = 0 (fresh start)

3.3 Multiple Instances vs Shared State

```
// Example showing instance isolation
@Component({
  selector: 'app-user-form',
  template: `
    <div class="form">
      <h3>User Form</h3>
      <input [(ngModel)]="userName" placeholder="Enter name">
      <p>You entered: {{ userName }}</p>
    </div>
  `
})
export class UserFormComponent {
  userName: string = ''; // Each instance has separate userName
}
```

```
<!-- Parent template with multiple instances -->
<div>
  <h2>Form Instance 1</h2>
  <app-user-form></app-user-form>
</div>
```

```
<h2>Form Instance 2</h2>
<app-user-form></app-user-form>
</div>
```

Output:

Result:

- Two completely separate forms appear
- Typing "Alice" in first form → Only first form shows "Alice"
- Typing "Bob" in second form → Only second form shows "Bob"
- Each instance has its own `userName` property
- No sharing of data between instances

4. Component Tree Structure

4.1 Understanding the Tree

Angular applications form a tree structure with the root component at the top and all other components as descendants (children, grandchildren, etc.).

```
AppComponent (Root)
|
├── HeaderComponent (Level 1)
|   |
|   ├── LogoComponent (Level 2)
|   |
|   └── NavigationComponent (Level 2)
|       |
|       ├── NavItemComponent (Level 3)
|       ├── NavItemComponent (Level 3)
|       └── NavItemComponent (Level 3)
|
```



```

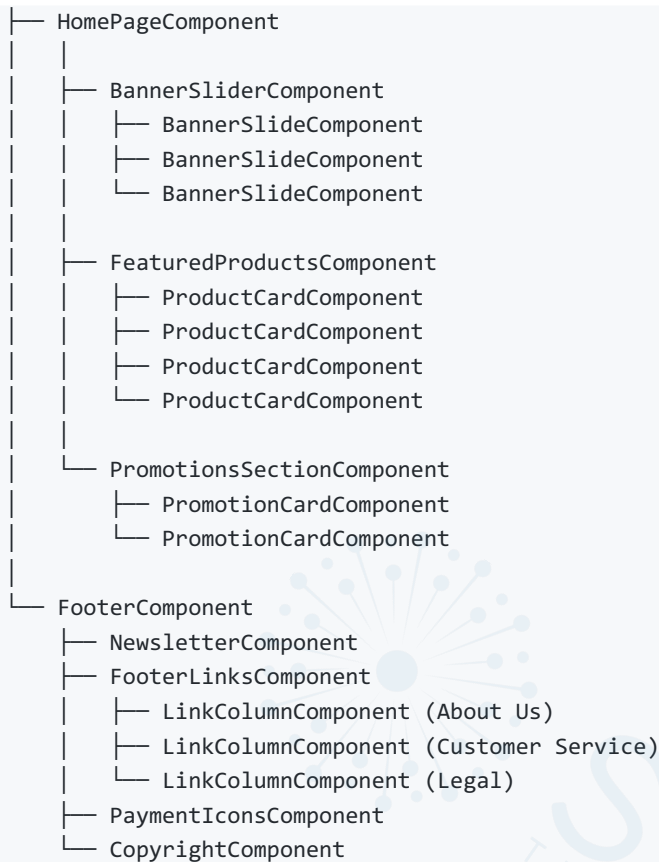
├─ MainComponent (Level 1)
│
│   └─ SidebarComponent (Level 2)
│       │
│       └─ MenuComponent (Level 3)
│           │
│           └─ MenuItemComponent (Level 4)
│               │
│               └─ MenuItemComponent (Level 4)
│                   │
│                   └─ MenuItemComponent (Level 4)
│
│   └─ UserWidgetComponent (Level 3)
│
│   └─ ContentComponent (Level 2)
│       │
│       └─ ArticleComponent (Level 3)
│           │
│           └─ ArticleHeaderComponent (Level 4)
│               │
│               └─ ArticleBodyComponent (Level 4)
│                   │
│                   └─ ArticleFooterComponent (Level 4)
│
│       └─ CommentSectionComponent (Level 3)
│           │
│           └─ CommentComponent (Level 4)
│               │
│               └─ CommentComponent (Level 4)
│                   │
│                   └─ CommentFormComponent (Level 4)
│
└─ FooterComponent (Level 1)
    │
    └─ ContactInfoComponent (Level 2)
        │
        └─ SocialLinksComponent (Level 2)
    
```

4.2 Tree Structure Benefits

Benefit	Description
Organization	Clear hierarchy makes codebase easy to navigate and understand
Encapsulation	Each component manages its own view and logic independently
Reusability	Components can be reused at different levels (MenuItemComponent used 3 times)
Maintenance	Changes to a component don't affect unrelated parts of the tree
Performance	Angular can optimize rendering by updating only changed branches
Change Detection	Angular tracks changes from root down through branches efficiently

4.3 Real-World Example: E-Commerce Application

```
// Complete e-commerce component structure
EcommerceAppComponent (Root)
├── TopBarComponent
│   ├── SearchComponent
│   │   ├── SearchInputComponent
│   │   └── SearchSuggestionsComponent
│   ├── CartIconComponent
│   └── UserAccountComponent
├── MainNavigationComponent
│   ├── CategoryMenuComponent
│   │   ├── CategoryItemComponent (Electronics)
│   │   ├── CategoryItemComponent (Clothing)
│   │   ├── CategoryItemComponent (Books)
│   │   └── CategoryItemComponent (Home & Garden)
│   └── DealsLinkComponent
```



4.4 Building the Tree Programmatically

Let's see how the tree is built through actual component code:

```

// Step 1: Root Component (app.component.ts)
@Component({
  selector: 'app-root',
  template: `
    <div class="app-wrapper">
      <app-header></app-header>
      <app-main-content></app-main-content>
      <app-footer></app-footer>
    </div>
  `
})
export class AppComponent { }

```

```
// Step 2: Header Component (header.component.ts)
```

```
@Component({
  selector: 'app-header',
  template: `
    <header>
      <app-logo></app-logo>
      <app-navigation></app-navigation>
      <app-user-menu></app-user-menu>
    </header>
  `
})
export class HeaderComponent { }
```

```
// Step 3: Navigation Component (navigation.component.ts)
```

```
@Component({
  selector: 'app-navigation',
  template: `
    <nav>
      <app-nav-item label="Home" link="/"></app-nav-item>
      <app-nav-item label="Products" link="/products"></app-nav-item>
      <app-nav-item label="About" link="/about"></app-nav-item>
      <app-nav-item label="Contact" link="/contact"></app-nav-item>
    </nav>
  `
})
export class NavigationComponent { }
```

```
// Step 4: Leaf Component (nav-item.component.ts)
```

```
@Component({
  selector: 'app-nav-item',
  template: `
    <a [href]="link">{{ label }}</a>
  `
})
export class NavItemComponent {
  @Input() label: string = '';
  @Input() link: string = '';
}
```

Output:

Tree Construction Process:

1. Angular bootstraps AppComponent (root)
2. AppComponent template contains `<app-header>`, `<app-main-content>`, `<app-footer>`
3. Angular creates instances of HeaderComponent, MainContentComponent, FooterComponent
4. HeaderComponent template contains `<app-logo>`, `<app-navigation>`, `<app-user-menu>`
5. Angular creates instances of LogoComponent, NavigationComponent, UserMenuComponent
6. NavigationComponent template contains 4 `<app-nav-item>` elements
7. Angular creates 4 instances of NavItemComponent
8. Tree is complete and rendered

4.5 Component Tree Best Practices

Practice	Recommendation	Reason
Depth	Keep tree depth reasonable (5-7 levels max)	Too deep makes debugging difficult and affects performance
Granularity	Create components for reusable UI pieces	Enables reusability and easier maintenance
Single Responsibility	Each component should have one clear purpose	Makes code easier to understand, test, and modify
Smart vs Presentational	Separate logic-heavy (smart) from display-only (presentational)	Improves testability and reusability
Naming	Use descriptive, hierarchical names	ProductCardHeaderComponent better than CardHeaderComponent

4.6 Tree Visualization in Angular DevTools

```
// Using Angular DevTools to inspect the tree
// Steps:
1. Open Angular application in Chrome
```

2. Open DevTools (F12)
3. Go to "Angular" tab
4. Select "Components" view
5. See visual tree of all components

// Example view:

```
AppComponent
├── HeaderComponent
│   ├── LogoComponent
│   └── NavigationComponent
│       ├── NavItemComponent
│       ├── NavItemComponent
│       ├── NavItemComponent
│       └── NavItemComponent
├── MainContentComponent (collapsed)
└── FooterComponent (collapsed)
```

Output:

DevTools Features:

- Click any component to inspect its properties
- View input/output bindings
- See component's current state
- Modify properties in real-time to test
- Understand parent-child relationships visually

5. Practical Example: Blog Application Tree

Let's build a complete blog application component tree to demonstrate all concepts:

```
// blog-app.component.ts (Root)
@Component({
  selector: 'app-blog',
  template: `
    <div class="blog-app">
      <app-blog-header></app-blog-header>
      <div class="blog-content">
        <app-sidebar></app-sidebar>
        <app-article-list></app-article-list>
      </div>
    </div>
  `
})
```

```
        <app-blog-footer></app-blog-footer>
      </div>
    `
  })
  export class BlogAppComponent { }
```

```
// blog-header.component.ts
@Component({
  selector: 'app-blog-header',
  template: `
    <header class="blog-header">
      <app-site-logo></app-site-logo>
      <app-main-nav></app-main-nav>
      <app-search-box></app-search-box>
    </header>
  `
})
export class BlogHeaderComponent { }
```

```
// article-list.component.ts
@Component({
  selector: 'app-article-list',
  template: `
    <main class="article-list">
      <h1>Recent Articles</h1>
      <app-article-card *ngFor="let article of articles">
      </app-article-card>
      <app-pagination></app-pagination>
    </main>
  `
})
export class ArticleListComponent {
  articles = [1, 2, 3, 4, 5]; // Simplified
}
```

```
// article-card.component.ts
@Component({
  selector: 'app-article-card',
  template: `
    <article class="article-card">
      <app-article-image></app-article-image>
    </article>
  `
})
export class ArticleCardComponent { }
```

```

    <app-article-title></app-article-title>
    <app-article-excerpt></app-article-excerpt>
    <app-article-meta></app-article-meta>
    <app-read-more-button></app-read-more-button>
  </article>
,
})
export class ArticleCardComponent { }

```

BlogAppComponent (Root)

```

|
├─ BlogHeaderComponent
|   ├─ SiteLogoComponent
|   ├─ MainNavComponent
|   │   ├─ NavLinkComponent (Home)
|   │   ├─ NavLinkComponent (Categories)
|   │   ├─ NavLinkComponent (About)
|   │   └─ NavLinkComponent (Contact)
|   └─ SearchBoxComponent
|
├─ SidebarComponent
|   ├─ CategoryListComponent
|   │   ├─ CategoryItemComponent (Technology)
|   │   ├─ CategoryItemComponent (Travel)
|   │   ├─ CategoryItemComponent (Food)
|   │   └─ CategoryItemComponent (Lifestyle)
|   ├─ RecentPostsComponent
|   │   ├─ MiniPostComponent
|   │   ├─ MiniPostComponent
|   │   └─ MiniPostComponent
|   └─ TagCloudComponent
|       └─ TagComponent (javascript)

```



```

|       |— TagComponent (angular)
|       |— TagComponent (tutorial)
|       |— TagComponent (webdev)
|
|— ArticleListComponent
|   |— ArticleCardComponent (Article 1)
|   |   |— ArticleImageComponent
|   |   |— ArticleTitleComponent
|   |   |— ArticleExcerptComponent
|   |   |— ArticleMetaComponent
|   |   |   |— AuthorBadgeComponent
|   |   |   |— DateBadgeComponent
|   |   |   |— ReadTimeBadgeComponent
|   |   |— ReadMoreButtonComponent
|   |— ArticleCardComponent (Article 2)
|   |   |— ... (same structure)
|   |— ArticleCardComponent (Article 3)
|   |   |— ... (same structure)
|   |— ArticleCardComponent (Article 4)
|   |   |— ... (same structure)
|   |— PaginationComponent
|   |   |— PaginationButtonComponent (Previous)
|   |   |— PageNumberComponent (1)
|   |   |— PageNumberComponent (2)
|   |   |— PageNumberComponent (3)
|   |   |— PaginationButtonComponent (Next)
|
|— BlogFooterComponent
|   |— NewsletterSignupComponent
|   |— SocialMediaLinksComponent
|   |   |— SocialLinkComponent (Twitter)
|   |   |— SocialLinkComponent (Facebook)

```

```
|   |— SocialLinkComponent (Instagram)
|   |— SocialLinkComponent (LinkedIn)
|— CopyrightComponent
```

Output:

Component Count in Blog App:

- Total unique component types: ~30
- Total component instances: ~100+ (counting all repetitions)
- Tree depth: 5 levels (Root → BlogApp → ArticleList → ArticleCard → ArticleMeta → AuthorBadge)
- Reusable components: NavLinkComponent (4 times), ArticleCardComponent (4 times), TagComponent (4 times), etc.
- Each instance operates independently with its own data

6. Summary

In this section, we explored nested components and the component tree:

- **Parent-Child Relationships:** Components can contain other components, creating a hierarchical structure
- **Component Rendering:** Element selectors (recommended), attribute selectors (directive-like), and class selectors (not recommended)
- **Component Instances:** Each use of a component creates an independent instance with its own state and lifecycle
- **Component Tree:** Applications form a tree structure starting from the root component, with all components as descendants
- **Instance Independence:** Multiple instances of the same component don't share state unless explicitly programmed
- **Lifecycle Management:** Components are created when rendered and destroyed when removed from the DOM

💡 Tip:

Next Steps: Now that you understand component hierarchy and nesting, you're ready to learn about component communication - how parent and child components share data using @Input, @Output, and other techniques.

This document is part of the Angular Development Course series. For questions or clarifications, please refer to the course materials or contact your instructor.

